Deep Agents 实战

第 14 讲：沙箱执行 — 安全运行代码



一张图看懂代码执行

三层能力：Backend、dcode 与托管沙箱

1. Python Backend



文件工具

SandboxBackendProtocol + execute

2. Deep Agents Code

本机 LLM 循环

远程工具调用

dcode --sandbox

3. LangSmith Sandboxes



托管执行环境

快照 · Auth Proxy · Mount

同一目标：安全地产生可审查、可下载的结果

沙箱 Backend：执行结果不等于命令成功

deepagents 0.7.22

普通 Backend 提供文件工具；SandboxBackendProtocol 额外提供 execute

普通 Backend

ls | read_file | write_file | edit_file | glob | grep

沙箱 Backend

+ execute

delete 仅在后端支持时可用；文件权限不约束 Shell 命令。

返回结构

ExecuteResponse

- **output：** 标准输出与错误输出
- **exit_code：** 0 成功；非零失败；None 未知
- **truncated：** 输出截断标志

判断依据

- 应用： `result.exit_code`
- Agent： `ToolMessage.artifact["exit_code"]`
- `status="success"` 只表示工具返回了结果

status="success" + exit_code=1 → 命令失败

大输出可能落盘；按提示分段读取。不要只凭消息状态判断程序成功。

隔离边界：保护什么，不保护什么

沙箱隔离宿主副作用，但不会自动让 Agent 或输入变得可信

沙箱有效保护



- 宿主文件、进程与环境变量
- 错误脚本、依赖安装与测试失败
- 执行环境的故障隔离

仍需额外治理



- 上下文注入：诱导执行危险命令
- 网络外传：HTTP / DNS 带走数据
- 不可信产物：代码、报告、构建物



不放秘密



限制网络



高风险 HITL



审查产物

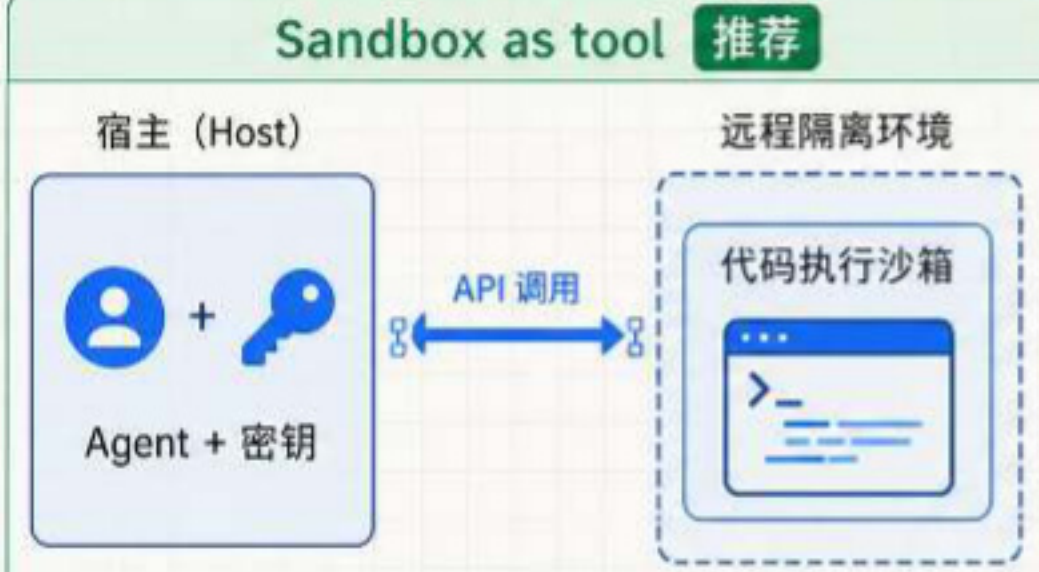


监控出站

两种集成模式

默认推荐 Sandbox as tool：Agent 与密钥留在宿主，代码在远程隔离环境执行

Sandbox as tool 推荐



优势

- 密钥不入沙箱
- 故障不丢 Agent 状态
- 可并行多沙箱

VS

Agent in sandbox



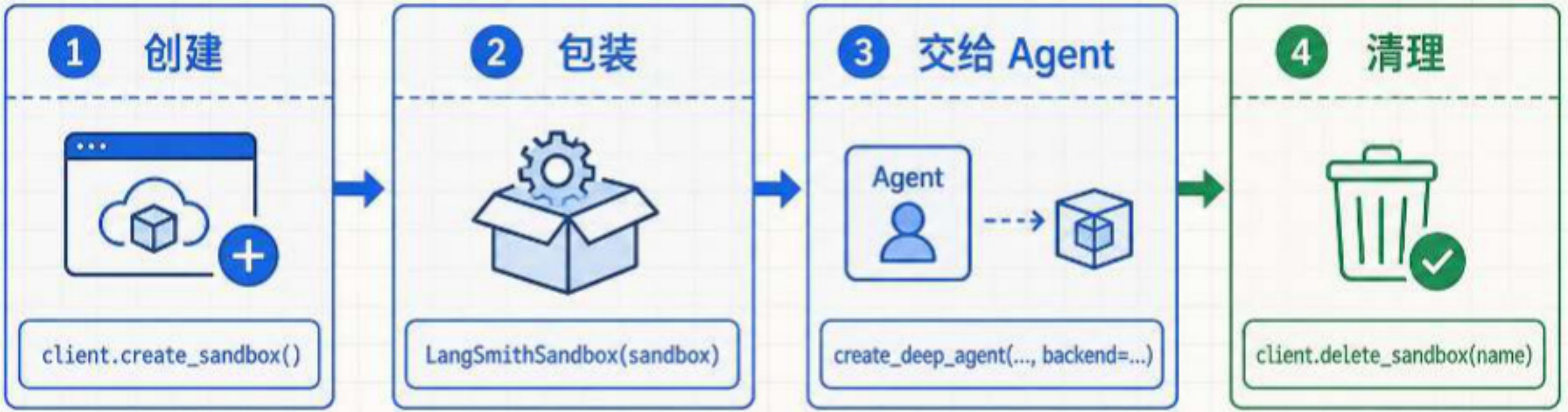
劣势

- 需重建镜像
- 额外通信层
- 凭证外传风险

仅在必须复现完整运行时、且 Provider 已妥善处理通信时再考虑

4 步接入远程沙箱

把 Provider 创建出的环境包装成 Backend，记得用 finally 或上下文管理器清理



运行前可用 backend.execute("python --version") 做健康检查；运行结束必须回收远程资源。

Provider 选择：先看语义，再看名称

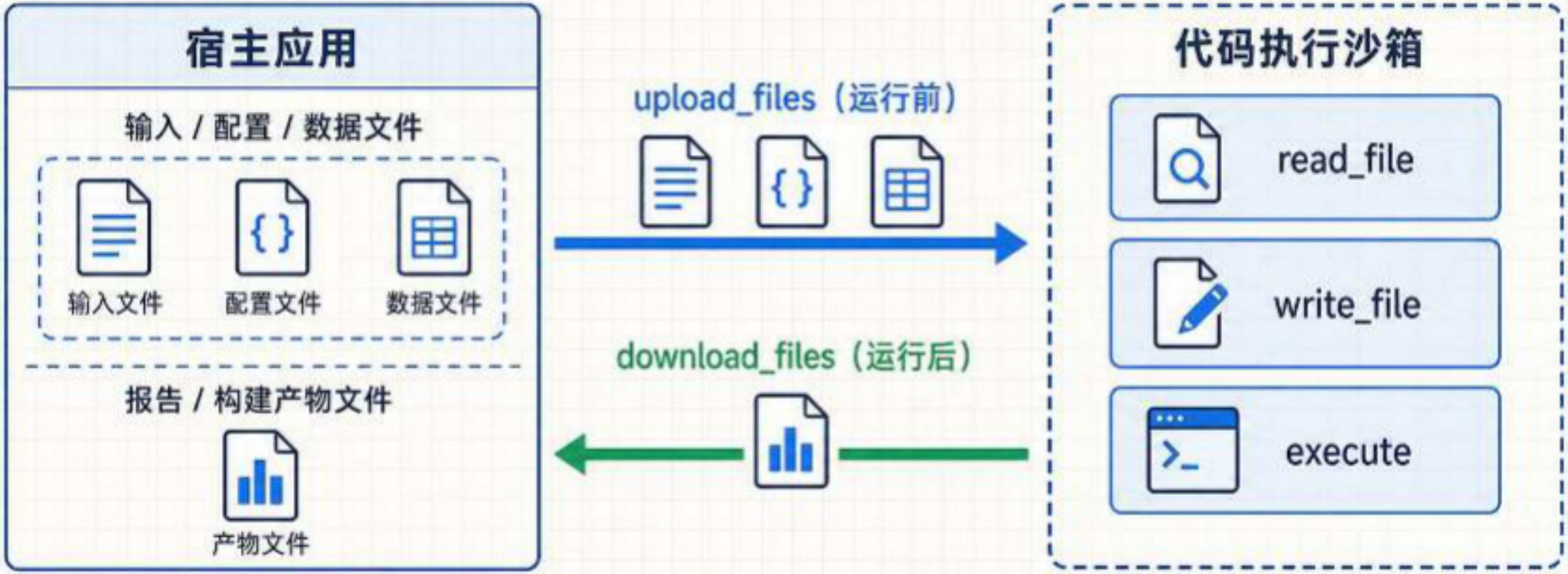
同一套工具语义，不代表相同的创建、清理、网络与凭证能力

运行区域	生命周期 / 回收	网络与凭证	镜像与依赖
LangSmith 快照 · Auth Proxy	AgentCore CodeInterpreter	Daytona 云端开发环境	E2B 第三方 Backend
Modal 容器 · 网络控制	NVIDIA OpenShell 自动清理	Runloop Devbox	Vercel Sandbox.create()

上线前逐一核对 Region、TTL、网络策略、凭证控制与镜像能力

文件的两个平面

Agent 工具只操作沙箱内部；宿主应用通过 Provider 原生 API 跨越边界



上传和下载是输入审查与产物审查的天然边界

生命周期与作用域：复用需要应用显式管理

thread_id / assistant_id 是映射依据，不会自动创建或绑定沙箱

Thread-scoped



- 同一 thread_id 查找并复用
- 不同任务使用不同环境

由应用维护 thread_id → sandbox

Assistant-scoped



- 跨对话保留仓库、依赖和缓存
- 共享状态需要权限与并发控制

由应用维护 assistant_id → sandbox

查找现有环境 → 不存在则创建 → 执行任务 → 按策略回收

⚠ TTL、快照与清理取决于 Provider 配置；例如 LangSmith 的 `idle_ttl_seconds`。不要假设所有 Provider 自动回收。

dcode 远程模式：本地循环，远程执行

启用 --sandbox 后，文件工具和 execute 定向到 Provider 沙箱



同名 Provider 优先级：配置声明 > 第三方 entry point > 内置 Provider

`--sandbox TYPE`

显式选择 Provider；裸 --sandbox 放在命令末尾

`--sandbox-id ID`

重连已有环境；跳过创建和清理

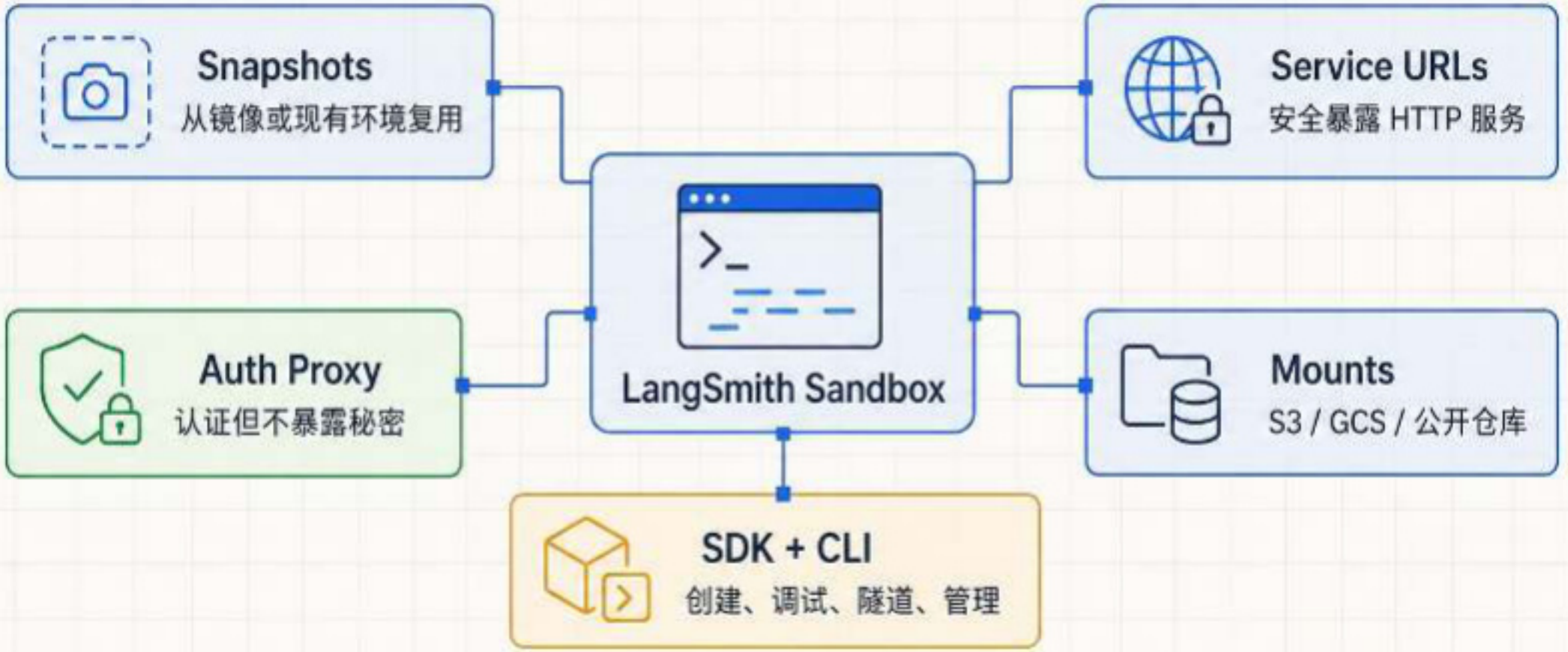
`--sandbox-snapshot-name NAME`

仅支持快照的 Provider；不能与 ID 同用

⚠ `--sandbox-setup` 会展开本地 `${VAR}`；只运行可信脚本，不要把长期凭证写入沙箱。

LangSmith 托管沙箱：不止是一个 Backend

Sandbox 既可被 Deep Agents 包装，也可通过 SDK、CLI 和托管资源独立使用



Auth Proxy：沙箱请求可以被认证，但 Agent 看不到长期秘密

两个扩展点：Backend 与 dcode Provider

Python 执行协议与 CLI 发现机制不能混为一谈

Backend 核对：deepagents 0.7.22

Python Agent

- 实现 SandboxBackendProtocol
- 继承 BaseSandbox 时必须实现：
  id · execute() · upload_files() · download_files()
- execute 返回 ExecuteResponse
  output · exit_code · truncated
- 没有 error 字段；命令失败用非零 exit_code
- 文件操作使用各自结果的 error

Deep Agents Code

- 实现 SandboxProvider
- 提供元数据、get_or_create()、delete()
- 注册 deepagents_code.sandbox_providers
  或通过 config.toml 声明
- 声明工作目录、ID 与快照能力

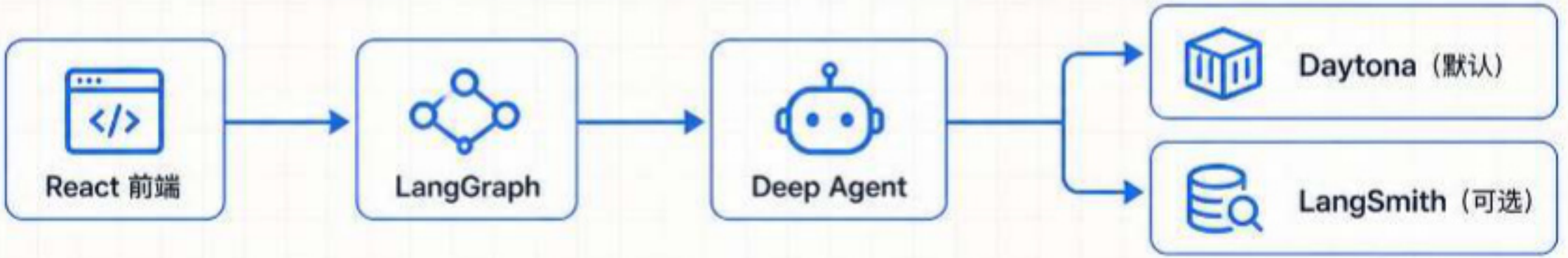
```
dcode --sandbox acme
```

⚠ class_path 会导入并执行本地 Python；只使用受信任模块。

AgentSeek 沙箱模板

一条命令跑起 Sandbox Coding Agent

```
>_ uv tool install agentseek-cli
>_  agentseek create deepagents/sandbox
```



1 .env 配置 .env

2 安装依赖

3 agentseek dev

隔离执行 Shell · 文件读写 · 流式工具调用 · 优雅退出自动清理

安全与可观测：形成闭环

任意一层都不足够；让限制、审批、Trace 与产物审查互相补位



默认原则：沙箱输出也是不可信输入；不得直接执行、发布或当作可靠事实。

生产就绪检查表

把“能跑”升级为“可控、可审查、可恢查”的执行系统

✓	**运行环境**	Provider 的区域、镜像、依赖与默认工作目录已确认
✓	**生命周期**	Thread / Assistant 作用域、TTL、快照和删除策略已定义
✓	**文件边界**	输入由 upload_files 审查；产物经 download_files 复核
✓	**安全策略**	秘密不入沙箱；网络、HITL、最小权限与出站监控已配置
✓	**可观测性**	Trace 可还原工具调用；异常输出与产物有处置路径

下一篇：文件权限 — 用声明式规则约束 Agent 的文件操作。